

CLOSURES

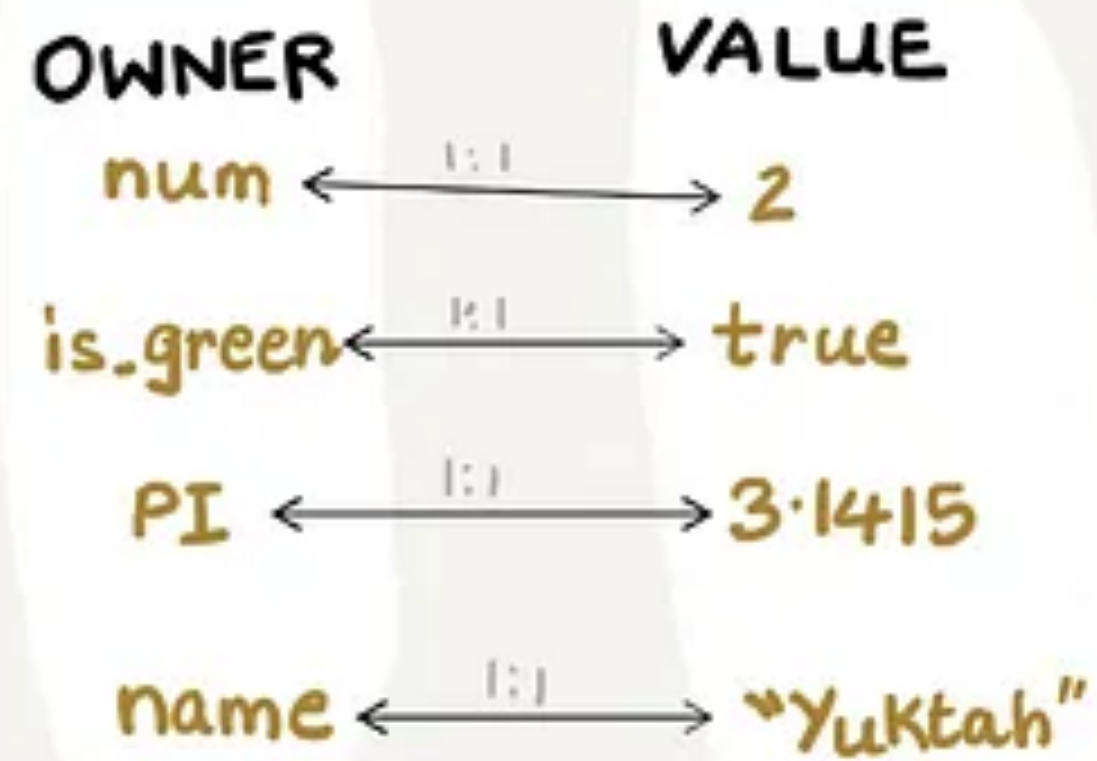
Definition

In Rust, closures are a powerful and flexible feature that allows you to define **anonymous functions** and **capture variables** from their surrounding **environment**.

A few reminder on ownership.. 😊

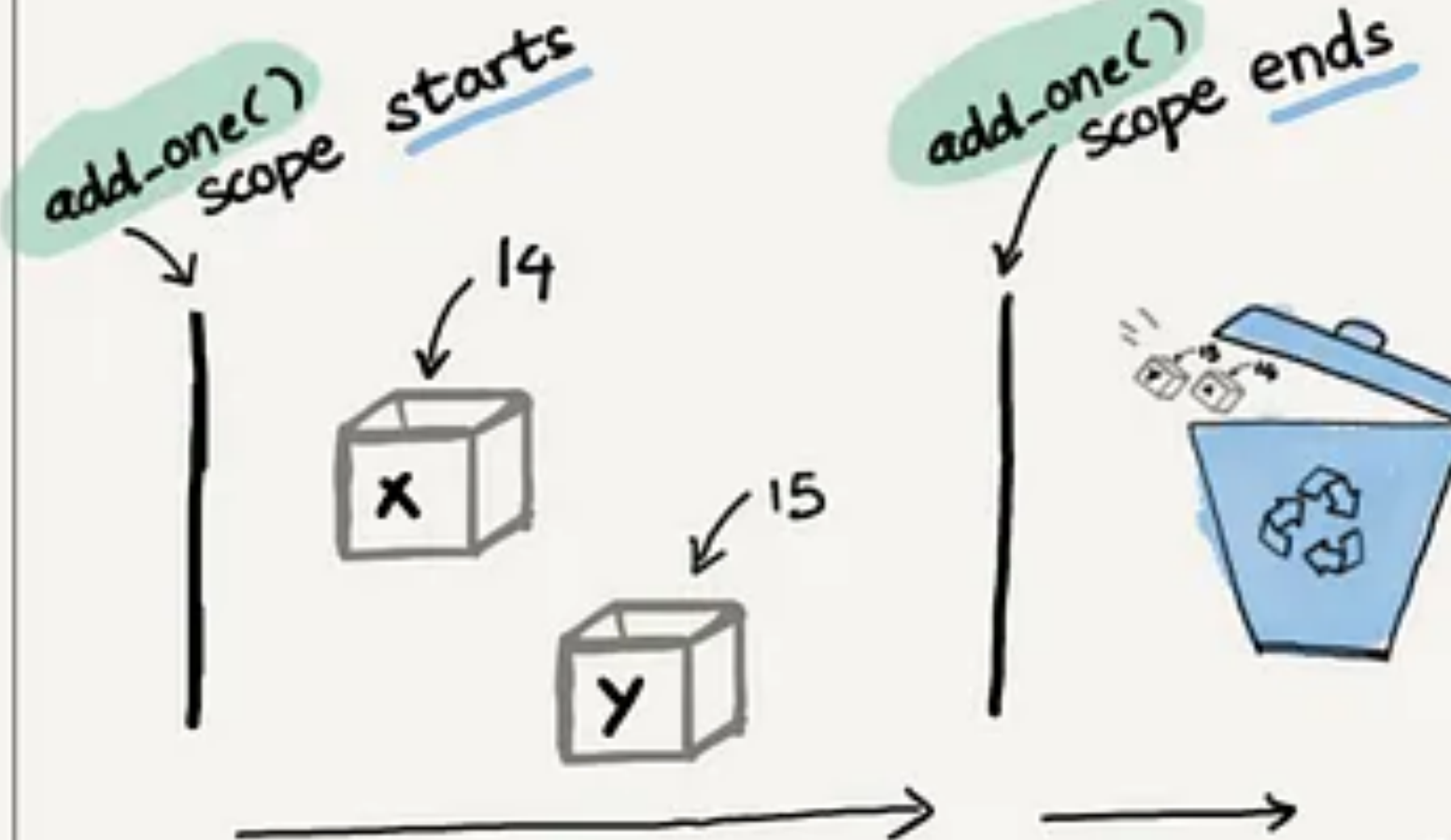
OWNERSHIP RULES

- 1 Each value in Rust has an owner
- 2 There can only be one owner at a time



- 3 When the owner goes out of scope, the value will be dropped

```
fn add_one(x: u32) → u32 {  
    let y = x + 1;  
    y  
}
```



Define a closure



```
let print_text = || println!("Defining Closure");
```



```
// call the closure  
print_text();
```

Closure with parameter



```
let add_one = |x: i32| x + 1;
```



```
// call the closure with value 2  
add_one(2);
```


Exercise 1: Closure basics

Fn, FnMut, FnOnce

The closure system is defined by **three core traits**—Fn, FnMut, and FnOnce—that determine **how closures interact with captured variables**, how many **times** they can be called, and whether they can **modify** their environment.

FnOnce: Indicates that a closure can be called once. After being called, the closure itself is consumed and can no longer be used.

FnMut: Indicates that a closure can be called multiple times and can modify captured variables when invoked.

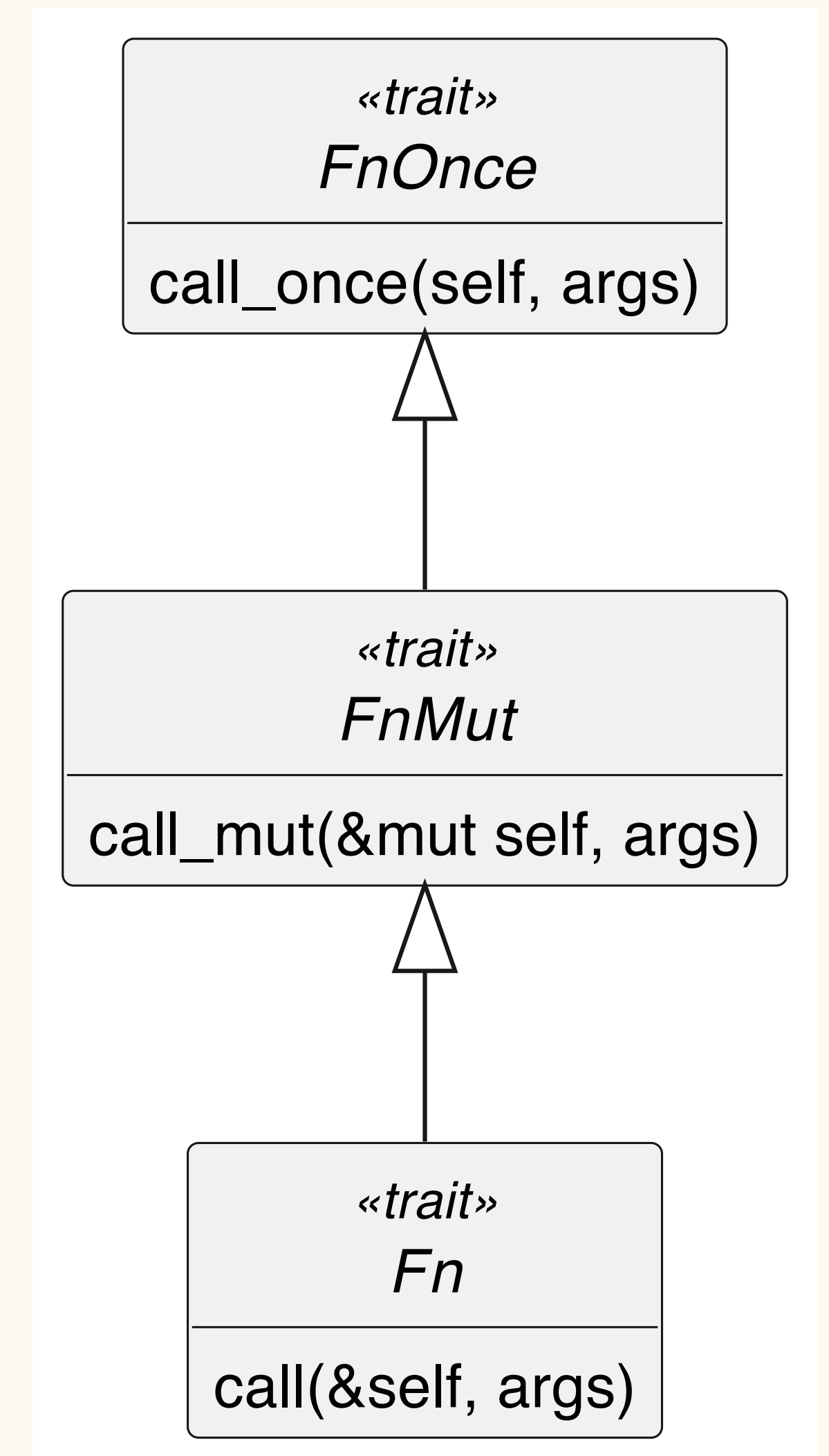
Fn: Indicates that a closure can be called multiple times and only reads captured variables without modifying them.

Inheritance

There is an **inheritance relationship** among these three traits:

-> **Fn inherits from FnMut, and FnMut inherits from FnOnce.**

-> So a closure implements Fn, it also automatically implements FnMut and FnOnce; if it implements FnMut, it also implements FnOnce.





```
pub trait Fn<Args>: FnMut<Args> {  
    fn call(&self, args: Args) -> Self::Output;  
}
```

Fn: The closure only reads the captured variable.



```
pub trait FnMut<Args>: FnOnce<Args> {  
    fn call_mut(&mut self, args: Args) -> Self::Output;  
}
```

FnMut: The closure modifies the captured variable.



```
pub trait FnOnce<Args> {  
    type Output;  
    fn call_once(self, args: Args) -> Self::Output;  
}
```

FnOnce: The closure moves the captured variable.

Closure implementing Fn



```
fn main() {  
    let s = String::from("hello");  
    let closure = || {  
        println!("{}", s); // Reads s without modification  
    };  
    closure(); // First call  
    closure(); // Second call  
}
```


Closure implementing FnMut

```
fn main() {  
    let mut s = String::from("hello");  
    let mut closure = || {  
        s.push_str(" world"); // Modifies s  
    };  
    closure(); // First call  
    closure(); // Second call  
    println!("{}", s); // Outputs "hello world world"  
}
```

Closure implementing FnOnce



```
fn main() {  
    let s = String::from("hello");  
    let closure = move || {  
        drop(s); // Moves s and drops it  
    };  
    closure(); // Called once  
    // closure(); // Error: Closure has been consumed  
}
```


Exercices 2-4: Closures (Fn, FnMut, FnOnce)

When to use each? 🤔

Trait	Use Case	Example
<code>FnOnce</code>	Called only once or needs to move captured variables	One-time operation that transfers ownership
<code>FnMut</code>	Called multiple times and modifies captured variables	A counter or updating state
<code>Fn</code>	Called multiple times and only reads captured variables	Logging or data queries

With functions

Just a quick reminder of what we have seen so far 😊

Function parameter for Fn

```
fn call_fn<F>(f: F)
where
    F: Fn(),
{
    f();
    f();
}

fn main() {
    let s = String::from("hello");
    call_fn(|| {
        println!("{}", s);
    });
}
```

Function parameter for FnMut

```
fn call_mut<F>(mut f: F)
where
    F: FnMut(),
{
    f();
    f();
}

fn main() {
    let mut s = String::from("hello");
    call_mut(|| {
        s.push_str(" world");
    });
    println!("{}", s); // Outputs "hello world world"
}
```

Function parameter for FnOnce



```
fn call_once<F>(f: F)
where
    F: FnOnce(),
{
    f();
}

fn main() {
    let s = String::from("hello");
    call_once(move || {
        drop(s);
    });
}
```


Your turn!

Closure Environment Capturing in Rust

Closure has a unique feature that allows it to capture the environment. This means the closure can use the values in its scope. For example,

```
fn main() {  
    let num = 100;  
  
    // A closure that captures the num variable  
    let print_num = || println!("Number = {}", num);  
  
    print_num();  
}
```

Output

```
Number = 100
```

Here, the closure bound to `print_num` uses the variable `num` which was not defined in it. This is known as closure environment capturing.

YOUR TEXT GOES HERE

YOUR TEXT GOES HERE

YOUR TEXT GOES HERE